

WAMU Uganda Super App

MVP Engineering Specification

Working name: WAMU

Tagline: Connect. Discover. Do More.

Initial market: Uganda

Primary platform: Android

Backend: Python + FastAPI

Database: PostgreSQL

Mobile: Flutter

1. MVP OBJECTIVE

The first WAMU release should prove one important hypothesis:

Can Ugandan users discover local businesses, communicate with them, purchase products/services and eventually complete transactions from one application?

The MVP should therefore focus on:

1. User accounts
2. Business accounts
3. Business discovery
4. Product/service listings
5. Search
6. Messaging
7. Orders
8. Notifications
9. Payment-provider integration layer
10. Administration

Do not build video, ride-hailing, maps, news and a full social network in version 1.

2. USER TYPES

WAMU initially has four major user roles.

Customer

Can:

- register
- search
- discover businesses
- view products
- chat
- order
- review
- save businesses
- receive notifications

Business

Can:

- create business profile
- add products
- receive messages
- receive orders
- manage orders
- see analytics
- respond to reviews

Administrator

Can:

- manage users
- verify businesses
- moderate content
- manage categories
- investigate reports
- monitor transactions
- suspend accounts

Platform/System

Responsible for:

- notifications
- search indexing
- background jobs
- fraud/risk checks

- AI
- analytics

3. APPLICATION STRUCTURE

The mobile application should initially have five main navigation areas.

```
WAMU
|
├─ Home
├─ Search
├─ Orders
├─ Messages
└─ Profile
```

Home contains:

```
Search
Categories
Nearby businesses
Popular businesses
Recommended products
Promotions
AI assistant
```

4. CUSTOMER JOURNEY

Registration

```
Open WAMU
  ↓
Enter phone number
  ↓
OTP verification
  ↓
Create profile
  ↓
Select interests/categories
```

↓
Home

Do not ask users for excessive information during registration.

5. BUSINESS REGISTRATION

Business signup
↓
Business name
↓
Category
↓
Phone
↓
Location
↓
Description
↓
Business images
↓
Submit
↓
Verification
↓
Business published

Businesses should have a clear verification status.

UNVERIFIED
PENDING
VERIFIED
SUSPENDED

6. BUSINESS PROFILE

Example:

FYNCAKES

★ 4.8 📍 Ntinda

Cakes | Bakery

About

Custom cakes for birthdays,
weddings and events.

[Message] [Order] [Directions]

Products

Birthday Cake

UGX 80,000

Wedding Cake

UGX 250,000

Chocolate Cake

UGX 60,000

Reviews

The business page becomes the digital storefront.

7. PRODUCT MODEL

A product should contain:

```
id
business_id
name
description
price
currency
category_id
images
stock_status
is_active
```

```
created_at
updated_at
```

Later:

```
variants
discounts
inventory
delivery_options
attributes
```

8. SEARCH

Search should support natural language.

Examples:

```
cake near Ntinda
restaurants in Kampala
phone shops
cheap shoes
plumber near me
wedding decorators
```

Initially:

```
PostgreSQL
  ↓
Search service
```

At scale:

```
PostgreSQL
  ↓
OpenSearch
```

↓
WAMU Search API

Search results should rank using:

relevance
distance
rating
popularity
availability
business verification

9. CHAT

Chat should support:

- one-to-one messages
- business/customer conversations
- text
- images
- voice notes later
- message status
- unread counts
- blocking/reporting

Architecture:

```
Flutter
|
WebSocket
|
FastAPI
|
Redis
|
Chat Service
|
PostgreSQL
```

Use WebSockets for real-time communication.

10. ORDERS

Customer:

```
Product
↓
Add to cart
↓
Checkout
↓
Delivery/pickup
↓
Payment
↓
Order created
```

Order states:

```
PENDING
CONFIRMED
PROCESSING
READY
OUT_FOR_DELIVERY
DELIVERED
CANCELLED
REFUNDED
```

11. PAYMENT ARCHITECTURE

Do not make the WAMU application directly hold customer funds initially.

Create an abstraction:

```
class PaymentProvider:
    def initiate_payment(self, amount, currency, reference):
        ...

    def check_status(self, reference):
        ...
```

```
def refund(self, reference):  
    ...
```

Then providers can implement:

```
MTN  
Airtel  
Card provider  
Other licensed providers
```

Architecture:

```
WAMU  
|  
▼  
Payment Service  
|  
├─ MTN Adapter  
├─ Airtel Adapter  
└─ Card Adapter
```

This means changing providers later doesn't require rewriting the entire application.

Payment webhooks must be verified, idempotent and logged.

12. DATABASE ARCHITECTURE

Use PostgreSQL.

Core tables:

```
users  
user_profiles  
businesses  
business_members  
business_categories  
categories  
products  
product_images  
orders  
order_items
```

```
payments
conversations
conversation_members
messages
reviews
locations
notifications
reports
favorites
sessions
audit_logs
```

Relationships:

```
User
|
├─ Profile
├─ Orders
├─ Conversations
├─ Reviews
└─ Favorites

Business
|
├─ Members
├─ Products
├─ Orders
├─ Reviews
└─ Location

Order
|
├─ User
├─ Business
├─ Items
└─ Payment
```

13. INITIAL DATABASE SCHEMA

users

```
id UUID PRIMARY KEY
phone VARCHAR UNIQUE
email VARCHAR NULL
status VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP
```

user_profiles

```
id UUID PRIMARY KEY
user_id UUID
first_name VARCHAR
last_name VARCHAR
username VARCHAR UNIQUE
avatar_url TEXT
bio TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

businesses

```
id UUID PRIMARY KEY
owner_id UUID
name VARCHAR
description TEXT
phone VARCHAR
category_id UUID
status VARCHAR
verification_status VARCHAR
latitude DECIMAL
longitude DECIMAL
created_at TIMESTAMP
updated_at TIMESTAMP
```

products

```
id UUID PRIMARY KEY
business_id UUID
name VARCHAR
description TEXT
price DECIMAL
currency VARCHAR
status VARCHAR
created_at TIMESTAMP
updated_at TIMESTAMP
```

orders

```
id UUID PRIMARY KEY
customer_id UUID
business_id UUID
status VARCHAR
subtotal DECIMAL
delivery_fee DECIMAL
total DECIMAL
currency VARCHAR
delivery_address TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

order_items

```
id UUID PRIMARY KEY
order_id UUID
product_id UUID
quantity INTEGER
unit_price DECIMAL
subtotal DECIMAL
```

14. API ARCHITECTURE

Use versioned APIs.

```
/api/v1/auth  
/api/v1/users  
/api/v1/businesses  
/api/v1/categories  
/api/v1/products  
/api/v1/search  
/api/v1/orders  
/api/v1/payments  
/api/v1/chat  
/api/v1/reviews  
/api/v1/notifications  
/api/v1/ai
```

Example:

```
POST /api/v1/auth/request-otp  
POST /api/v1/auth/verify-otp  
  
GET /api/v1/businesses  
GET /api/v1/businesses/{id}  
  
POST /api/v1/businesses  
  
GET /api/v1/products  
POST /api/v1/products  
  
POST /api/v1/orders  
GET /api/v1/orders/{id}  
  
POST /api/v1/payments  
GET /api/v1/payments/{id}
```

15. FASTAPI PROJECT STRUCTURE

```
wamu-backend/  
|  
├─ app/  
|   └─ main.py  
|   |  
|   └─ core/  
|       └─ config.py  
|       └─ security.py
```

```

├── database.py
├── logging.py
├── api/
│   ├── v1/
│   │   ├── auth.py
│   │   ├── users.py
│   │   ├── businesses.py
│   │   ├── products.py
│   │   ├── search.py
│   │   ├── orders.py
│   │   ├── payments.py
│   │   ├── chat.py
│   │   └── reviews.py
│   ├── models/
│   │   ├── user.py
│   │   ├── business.py
│   │   ├── product.py
│   │   ├── order.py
│   │   ├── payment.py
│   │   └── message.py
│   ├── schemas/
│   ├── services/
│   │   ├── auth.py
│   │   ├── business.py
│   │   ├── order.py
│   │   ├── payment.py
│   │   ├── search.py
│   │   └── notification.py
│   ├── repositories/
│   ├── workers/
│   ├── integrations/
│   │   ├── payments/
│   │   ├── maps/
│   │   └── notifications/
├── migrations/
├── tests/
├── Dockerfile
├── docker-compose.yml
├── pyproject.toml
└── README.md

```

Keep business logic out of the route handlers.

16. FLUTTER STRUCTURE

```
wamu-mobile/  
├── lib/  
│   ├── main.dart  
│   ├── core/  
│   │   ├── network/  
│   │   ├── storage/  
│   │   ├── theme/  
│   │   └── routing/  
│   ├── features/  
│   │   ├── auth/  
│   │   ├── home/  
│   │   ├── search/  
│   │   ├── businesses/  
│   │   ├── products/  
│   │   ├── cart/  
│   │   ├── orders/  
│   │   ├── chat/  
│   │   ├── profile/  
│   │   └── ai/  
│   └── shared/  
│       ├── widgets/  
│       ├── models/  
│       └── utils/  
└──
```

Use feature-based organization rather than putting every screen into one giant folder.

17. AUTHENTICATION

Recommended initial flow:

```
Phone number  
  ↓  
OTP
```

↓
Access token
↓
Refresh token

Access tokens should be short-lived.

Refresh tokens should be securely stored.

Never store sensitive authentication information in plain text.

18. ADMIN DASHBOARD

Create a separate web application.

Recommended:

React/Next.js

or initially a lightweight frontend connected to FastAPI.

Dashboard:

```
-----  
WAMU ADMIN  
-----  
  
Users           12,450  
Businesses      1,230  
Orders          5,820  
Transactions    UGX XXX  
  
-----  
  
Users  
Businesses  
Verification  
Reports  
Orders  
Payments  
Categories  
Analytics
```

System Health

19. MODERATION

Every important user-generated object should support reporting.

User
Business
Product
Message
Review
Content

Example:

```
POST /api/v1/reports
```

Report:

```
reporter_id  
target_type  
target_id  
reason  
description  
status  
created_at
```

20. NOTIFICATION SERVICE

Events:

```
NEW_MESSAGE  
NEW_ORDER  
ORDER_CONFIRMED  
ORDER_READY  
PAYMENT_SUCCESS  
PAYMENT_FAILED
```

```
NEW_REVIEW
BUSINESS_VERIFIED
SECURITY_ALERT
```

Architecture:

```
Application
  ↓
Event
  ↓
Queue
  ↓
Notification Worker
  ↓
Push Notification
```

This keeps notification processing out of the main request.

21. BACKGROUND JOBS

Use background workers for:

- notifications
- emails
- image processing
- search indexing
- analytics
- AI processing
- payment reconciliation
- scheduled jobs

Initially:

```
FastAPI
+
Redis
+
Celery
```

or another suitable Python task queue.

22. MEDIA

Do not store photos/videos directly in PostgreSQL.

Use:

```
Mobile
↓
Upload API
↓
Object Storage
↓
CDN
```

Store only:

```
file_id
url/key
owner_id
mime_type
size
metadata
```

in PostgreSQL.

23. AI ARCHITECTURE

The WAMU AI layer should have three levels.

Level 1 — Search assistant

```
User:
"Find cakes near Ntinda"

AI
↓
Search Wamu database
↓
Return businesses
```

Level 2 — Recommendation

```
User preferences
+
Search history
+
Business data
+
Location
↓
Recommendation engine
```

Level 3 — Agent

Eventually:

```
User
↓
AI
↓
Search
↓
Compare
↓
Ask confirmation
↓
Order
↓
Payment
```

The AI should never silently execute consequential actions.

24. AI DATA ARCHITECTURE

```
WAMU DATABASE
|
▼
Search / Retrieval
|
▼
Relevant business/product data
```



For factual local recommendations, prefer WAMU's own structured data over allowing the model to invent businesses or prices.

25. MAPS AND LOCATION

Store locations using latitude/longitude.

PostgreSQL can later use PostGIS.

Example:

```
business
latitude
longitude
```

Then:

```
User location
  ↓
Nearby search
  ↓
Businesses within radius
```

This allows:

```
"Businesses within 2 km."
```

26. PERFORMANCE TARGETS

For MVP, target:

API response:
< 500 ms for common requests

Search:
< 1 second

Chat:
near real-time

App startup:
as fast as practical on entry-level Android devices

Do not optimize prematurely.

Measure first.

27. OBSERVABILITY

Every production system should have:

Application logs
Error tracking
Metrics
Health checks
Audit logs
Database monitoring

Useful endpoints:

GET /health
GET /health/database
GET /health/redis

28. SECURITY

Minimum requirements:

```
HTTPS
JWT/session security
OTP rate limiting
API rate limiting
Input validation
SQL injection protection
Secure file uploads
Authorization checks
Audit logging
Secret management
Encrypted sensitive data where appropriate
```

Most importantly:

Never trust the mobile application.

Every permission must be checked on the backend.

For example, don't assume:

```
Flutter says:
"I own this business."
```

The backend must verify it.

29. DEVOPS

Development:

```
Docker
Docker Compose
Git
GitHub
```

CI:

```
Push code
↓
Tests
↓
Lint
```

```
↓  
Build  
↓  
Security checks  
↓  
Deploy staging
```

Production:

```
Internet  
↓  
Cloud Load Balancer  
↓  
Nginx  
↓  
FastAPI  
↓  
PostgreSQL  
↓  
Redis
```

Use managed database infrastructure when the budget allows.

30. ENVIRONMENTS

Maintain three environments:

```
development  
staging  
production
```

Never develop directly against production.

Example:

```
api-dev.wamu...  
api-staging.wamu...  
api.wamu...
```

31. TESTING

Backend:

```
pytest
```

Tests:

```
Unit tests
Integration tests
API tests
Database tests
Payment tests
Authentication tests
Permission tests
```

Flutter:

```
Unit tests
Widget tests
Integration tests
```

Critical workflows must have automated tests.

32. MVP DEVELOPMENT ORDER

Do not build randomly.

Build in this order:

Sprint 1

```
Project setup
Docker
PostgreSQL
FastAPI
Flutter
CI/CD
```

Sprint 2

Authentication
Users
Profiles

Sprint 3

Businesses
Categories
Business profiles

Sprint 4

Products
Product images
Search

Sprint 5

Chat
WebSockets
Notifications

Sprint 6

Cart
Orders
Order management

Sprint 7

Payment abstraction
Payment provider integration
Webhook handling

Sprint 8

Reviews
Reports

Admin dashboard
Analytics

Sprint 9

AI search
Recommendation system

Sprint 10

Security audit
Performance testing
Bug fixing
Production deployment

33. MVP RELEASE

The first release should answer:

Can people register?
↓
Can they find businesses?
↓
Can businesses list products?
↓
Can customers communicate?
↓
Can customers order?
↓
Can transactions be completed?
↓
Will customers return?

Those answers are more important than having 100 features.

34. FIRST TARGET MARKET

Do not launch across Uganda immediately.

Start with:

Kampala

Then concentrate on selected areas.

Potential categories:

```
Food
Cakes
Fashion
Beauty
Electronics
Restaurants
Home services
Events
Retail
```

This gives WAMU a dense marketplace.

35. FIRST 1,000 BUSINESSES

Instead of waiting for businesses to discover WAMU:

Build a business acquisition team.

Approach:

```
100 businesses
  ↓
10 categories
  ↓
Kampala
  ↓
Free onboarding
  ↓
Product catalogue creation
  ↓
First orders
```

The objective is to make WAMU useful before charging heavily.

36. FIRST 10,000 USERS

Don't advertise everywhere.

Create concentrated communities.

For example:

```
University students
Young professionals
Small businesses
Online sellers
Creators
```

Give users a reason to invite others.

Example:

```
Customer joins
  ↓
Discovers business
  ↓
Shares business
  ↓
Friend joins
  ↓
Friend buys
  ↓
Business invites customers
```

This creates the network effect.

37. GROWTH LOOP

The ideal loop:

```
More businesses
  ↓
More products/services
  ↓
More customers
```

↓
More transactions
↓
More business revenue
↓
More businesses join
↓
More customers

This is what you should optimize—not just downloads.

38. KEY METRICS

Don't obsess over downloads.

Track:

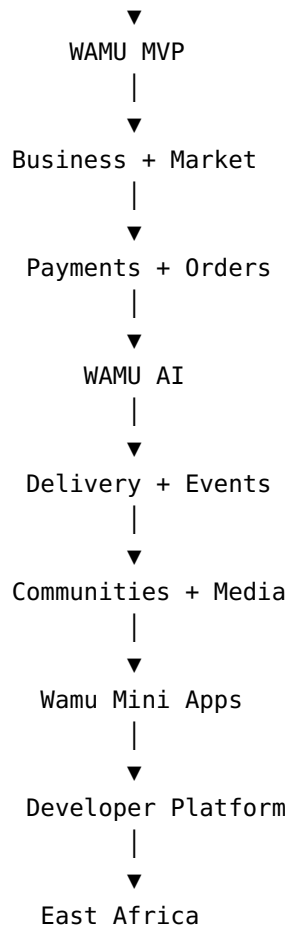
DAU
MAU
Retention
Businesses onboarded
Verified businesses
Orders
GMV
Successful payments
Messages
Searches
Search → business conversion
Business → order conversion
Customer retention
Cost of acquiring a user
Revenue per business

The most important early metric may be:

How many users successfully complete a meaningful local transaction?

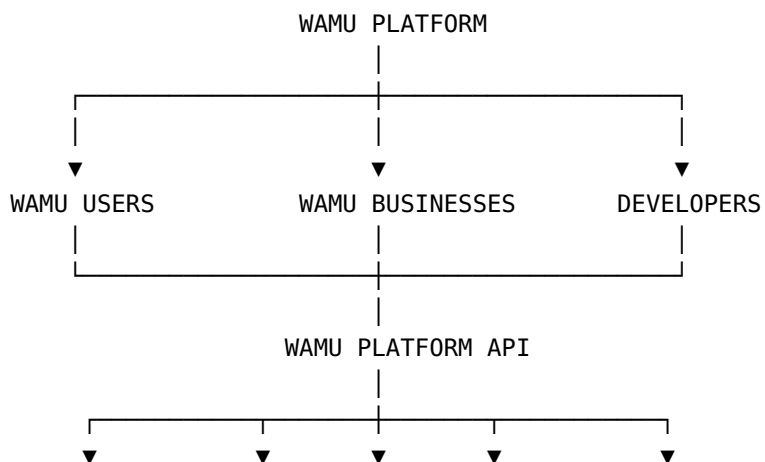
39. WAMU ROADMAP

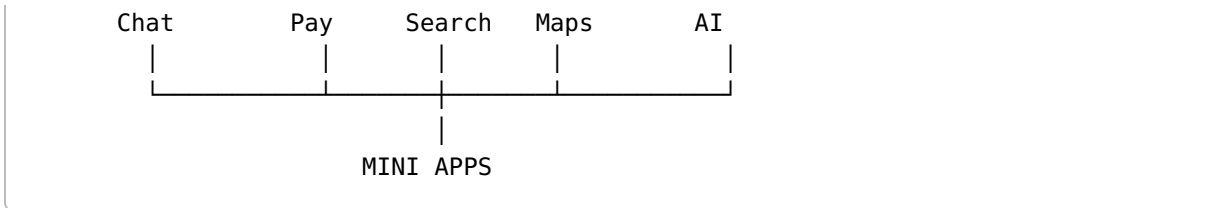
2026
|



40. THE LONG-TERM PLATFORM

Eventually the architecture becomes:





At that point WAMU isn't simply an application.

It becomes **infrastructure for digital services in Uganda.**

41. WHAT I WOULD BUILD FIRST

If you and I were starting the codebase tomorrow, our first repository would be:

```
wamu/  
├─ wamu-backend/  
├─ wamu-mobile/  
├─ wamu-admin/  
├─ infrastructure/  
└─ documentation/
```

And the first working milestone would be:

```
User  
↓  
Register  
↓  
Login  
↓  
Home  
↓  
Search business  
↓  
Open business  
↓  
View product  
↓  
Message business  
↓  
Add product  
↓  
Place order
```

Once that flow works end-to-end, we have the **foundation of WAMU**.

Everything else can be added around it.